# Data Source Decides Forgetting: Self-Generated Rollouts vs. Human Demonstrations for a Flow-Matching VLA

**Ende Shen**
Independent Researcher
endeshen99@gmail.com

**Abstract:** Continually self-improving robots must acquire new skills without forgetting the ones they have. Recent work has shown that on-policy reinforcement learning (RL) forgets far less than supervised fine-tuning (SFT) when updating vision–language–action (VLA) models; for language models, this has been attributed to the data RL trains on, sampled from the model's own distribution, rather than to its objective. We test the effects of on-policy data on a flow-matching VLA ($\pi_{0.5}$). Starting from a skill just improved by on-policy RL, we fine-tune on new tasks with SFT, holding everything fixed except the training data: successful rollouts of the policy itself, or human demonstrations. Across three new tasks, human demonstrations learn the new task but erase the prior skill, while the policy's own rollouts keep it on all three and modestly improve success on one of the new tasks. On one task pair, a full-precision replication shows that on-policy RL and SFT on the same rollouts learn the new task comparably and both keep the prior skill, while demonstrations erase it. Our results suggest that for flow-matching VLAs the data source, more than the objective, decides whether prior skills survive, and that SFT on the policy's own successes protects them.

**Keywords:** continual learning, self-improvement, vision-language-action models, forgetting

## 1 Introduction

Robot foundation models have progressed to show broad generalization, skill transfer across embodiments, and learning of new tasks from a few demonstrations [1, 2, 3, 4, 5, 6]. Commercial deployment tends to start narrow and widen over time [7, 8, 9, 10], with the policy continually updated in the process [11, 12]. Updating the policy, however, risks forgetting prior skills [13, 14]: fine-tuning $\pi_{0.5}$, a flow-matching VLA, on demonstrations of a new task has been shown to erase a skill it previously performed [15, 16]. Yet VLAs can also learn a sequence of tasks by on-policy RL with LoRA and little forgetting [17]. What do the updates that avoid forgetting have in common?

Research on language models suggests that on-policy RL forgets less than supervised fine-tuning (SFT) because of the data it trains on, sampled from the model's own distribution, rather than its objective: SFT on data an RL-improved model generated forgets as little as RL itself [18, 19]. Several VLA methods already rely on a version of this mechanism, training on a policy's own rollouts, alone [17] or mixed into demonstration fine-tuning [15], to protect prior capabilities, or attributing their gains to data that stays close to the base policy [20]. These methods establish that self-generated data helps, but none isolates the contribution of the data under a fixed objective, or compares objectives on the same self-generated data.

Project page: https://endeshen.com/blog/2026/what-forgetting-looks-like/.

Workshop on Continually Self-Improving Robots (CSIR) at the 10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

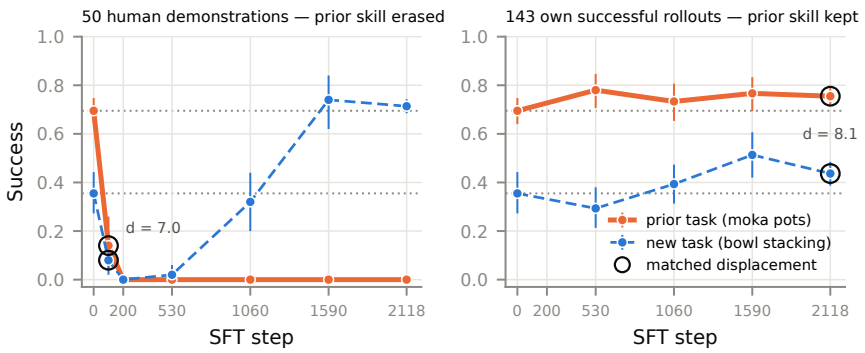



Figure 1: Under one SFT objective, the data source decides whether the prior skill survives. **Left:** 50 human demonstrations erase the prior skill (moka pots) within 200 steps, before the new task (bowl stacking) is learned. **Right:** the policy's own successful rollouts keep it throughout; the new task improves modestly. At step 100 the demonstration run has moved as far from Start as the own-rollout runs ever do (displacement 7.0 vs. 8.1, Table 2) and has already lost the skill; $d$: displacement from Start, $\times 10^{-3}$. Points pool seeds with 95% CIs over initial states (evaluation sizes in Table 9).

We ask whether the policy's own rollouts forget less than human demonstrations when the objective is held fixed. Starting from a $\pi_{0.5}$ checkpoint that on-policy RL has improved on one LIBERO task (success 0.55 to 0.70), we fine-tune on a second task from a different suite with identical optimizer, training budget, and initialization, and repeat the comparison on two further new tasks. The conditions differ only in where the training actions come from: (i) successful rollouts generated by the policy itself, or (ii) 50 human demonstrations. For each we measure success on the new task and on the prior task.

To probe the mechanism, we track how far each update moves the policy from its starting point (its displacement), using a likelihood-free action-space distance in place of the KL divergence that predicts forgetting in language models [18]. To ablate the objective, we run, on the same rollouts, the on-policy RL recipe that produced the prior skill. Under the paper's training precision it barely moves the policy; at full precision on one task pair, RL and SFT on the same rollouts learn the new task comparably and both keep the prior skill, while demonstrations still erase it (Section 4.4).

Figure 1 shows the main result, and our findings are:

- *The data source decides retention.* With objective, optimizer, budget, and initialization fixed, human demonstrations raise new-task success from 0.40 to 0.71 and drive prior-task success from 0.70 to 0.00, and nearly erase the prior skill on all three new tasks (0.00–0.05). The policy's own successful rollouts of the new task, with no prior-task data in the batch, keep it on all three (0.69–0.80). They learn the new task modestly on bowl stacking (+6 points, 6 runs) and leave it unchanged on the other two (Table 3).
- *Forgetting precedes learning.* Under demonstrations both tasks first collapse and only the new one recovers: the prior skill falls to 0.14 after 100 gradient steps and 0.00 after 200, before new-task success has recovered, so no stopping rule on the new task, and no lower learning rate, can protect it.
- *Displacement tracks forgetting but does not decide it.* Displacement separates retained from forgotten policies (Figure 3), but at matched displacement demonstrations have already erased the skill and own rollouts have not. Under the same observations, demonstration actions differ from the policy's own by roughly ten times as much as its rollouts do (Section 4.5), a property of the data that can be measured before training and may predict whether a prior skill survives.
- *The policy's own rollouts are the protective ingredient.* Mixed into demonstrations at about a third of the training frames, they keep the prior skill (0.70) at the demonstrations' level of

learning (0.69 vs. 0.71). The same share of human prior-task demonstrations keeps it only partly (0.59).

Replay of the policy's own rollouts, self-demonstrations, and on-policy RL keep prior skills for the same reason: the update trains on actions the policy already produces, and that, more than the objective or the distance moved, decides whether a self-improved skill survives.

## 2 Related Work

**Forgetting in VLAs.** Catastrophic forgetting is well documented in VLAs: fine-tuning on demonstrations of a new task degrades skills the policy already had [16, 21]. Recent results are more encouraging for pretrained models: Liu et al. [22] find that pretrained VLAs resist forgetting when earlier data is replayed, and that skills which appear lost recover quickly on re-finetuning. However, the mechanism of forgetting, and what protects against it, is still being worked out. Hu et al. [17] show that a VLA can learn tasks sequentially by on-policy RL with LoRA adapters and forget little; Garg et al. [15] recover a lost skill by mixing a frozen base policy's own prior-task rollouts into demonstration fine-tuning, a recipe our Demo+Own condition runs under our controls; and RECALL [23] finds that even actively collected recovery demonstrations erase prior behavior, mitigated only partly by replay or elastic weight consolidation. None of these isolates the data source under a fixed objective or measures the time course of forgetting.

**The language-model account.** In language models the lower forgetting of on-policy RL has been traced to its data: forgetting is predicted by the divergence between fine-tuned and initial policies, and SFT on data an RL-improved model generated forgets as little as RL [18]. Chen et al. [19] find that approximately on-policy SFT retains prior capabilities while SFT on samples from the initial policy does not. Our own-rollout condition adds a success filter to the latter and retains. This may be because of the filter or our short training budget, and we state it as a hypothesis. Chen et al. [19] also find that for matched gains on-policy SFT still forgets somewhat more than RL, which they attribute to RL's mode-seeking. TMS [24] defines a policy–label divergence and reports it predicts forgetting. Our off-policyness measure (Section 4.5) is its analogue for continuous actions. Self-distillation [25, 26, 27, 28] and on-policy replay [29] make on-policy data the method but need a teacher, a KL anchor, or prior-task prompts. We ask whether the policy's own filtered rollouts of the new task, with no teacher, no KL anchor, and no prior-task data, already preserve a prior skill.

**Replay and mixed data.** Experience replay [30] and pseudo-rehearsal [31, 32] protect prior tasks by training on data from the model's own distribution. Practical robot pipelines add human corrections at failure states [33, 34]. Our own-rollout condition keeps the prior skill with no prior-task data in the training set at all, and our two replay conditions test a single replay fraction (about a third of the frames) with two kinds of replayed prior-task data, the policy's own rollouts or human demonstrations.

## 3 Setup

**Policy and starting point.** We use $\pi_{0.5}$ [1] in the LIBERO simulator. *Start*, the initial policy for every condition, is the released `pi05_libero` checkpoint improved by on-policy RL on one LIBERO-10 task, *put both moka pots on the stove* (task 8, *moka pots* below): 20 iterations of GRPO with 64 rollouts each, using $\pi$RL's Flow-SDE sampler [35] ($\sigma = 0.5$). Under the evaluation protocol, with deterministic decoding (ODE; Table 12), success on moka pots rose from 0.55 to 0.70 on the same episodes ($p$ =0.002; Table 9). This is the self-improved skill whose survival we measure.

**New tasks.** From Start, we fine-tune on a new LIBERO-90 task. The first is *stack the right bowl on the left bowl and place them in the tray* (task 64, *bowl stacking*), a different scene and suite from moka pots. Start already succeeds on it 0.35 of the time, which is what makes filtering its own rollouts for successes possible. Two further LIBERO-90 tasks repeat the Own and Demo conditions

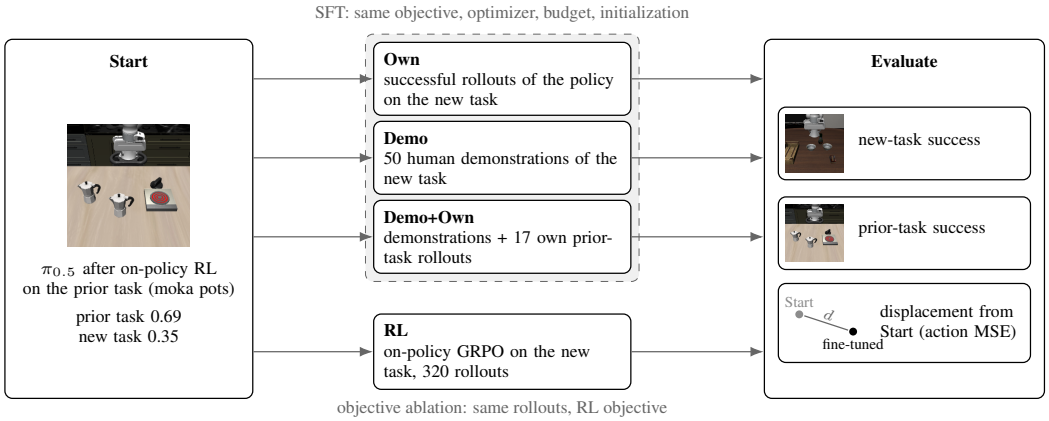



Figure 2: Design. Every condition starts from Start, a $\pi_{0.5}$ checkpoint improved by on-policy RL on the prior task (*put both moka pots on the stove*, LIBERO-10), and is fine-tuned on a new task (for example, *stack the right bowl on the left bowl and place them in the tray*, LIBERO-90). The three SFT conditions share everything but the training data. RL is the objective ablation. Each condition is scored on the new task, the prior task, and its displacement from Start.

against the same prior task: *put the right moka pot on the stove* (task 38, *one moka pot*) and *pick up the black bowl on the left and put it in the tray* (task 60, *bowl to tray*; Section 4.6). All updates train only the action expert, with the vision–language backbone frozen.

**Conditions.** Every supervised condition minimizes the same flow-matching SFT loss from Start with identical optimizer settings and a fixed budget of 2118 gradient steps (Appendix B). A larger training set therefore means fewer passes over the data, not more updates. All main-text runs train in bfloat16, not at full precision (Appendix B). The supervised conditions differ only in their training set (Table 4). Figure 2 summarizes the design: every condition starts from Start and is scored on the new task, the prior task, and displacement.

- **Own.** Filtered behavior cloning on the policy's own successful rollouts of the new task, collected with the stochastic training sampler, with no prior-task episodes. Six runs: three seeds on each of two collection passes of 320 rollouts (154 and 143 successes; Table 4).
- **Demo.** The 50 LIBERO-90 demonstrations of the new task, re-rendered at 256 px to match the evaluation images. Three seeds.
- **Demo+Own.** Demo's demonstrations (256 px) plus replay of the prior task: 17 of Start's own successful moka-pots rollouts (17 of 67 episodes, about a third of the training frames). Unlike Own, the policy's data here is prior-task replay, not new-task rollouts.
- **RL.** The recipe that produced Start, continued on the new task: 5 GRPO iterations of 64 rollouts. RL is repeated at full precision on the bowl-stacking pair, alongside Own and Demo for comparison (Section 4.4, Appendix G).

**Controls and variants.** Variants of each condition (resolution, learning rate, data size, a second self-generation round, RL budget) are listed in Appendix F.

**Evaluation protocol.** A checkpoint is evaluated on a task over 25 initial states $\times$ 8 initial-noise seeds (200 episodes); intermediate checkpoints use 2 seeds (50), and the main comparisons in Table 1 use two sets of initial states where available (400). Every policy sees the same states and seeds, so episodes are paired across policies, and every comparison is against Start on the same episodes. Episodes end at success, as judged by LIBERO's task predicate, or after 520 control steps on moka pots and 400 on the LIBERO-90 tasks. Decoding is deterministic: the ODE sampler with 10 flow

Table 1: **Retention under one objective.** Final checkpoints after updating from Start, 200–400 paired episodes each; Own pools six runs, Demo three (per-run values in Appendix F). $\Delta$: paired difference from Start with a 95% bootstrap CI over initial states, computed on the episodes each row shares with Start, so it can differ slightly from the difference between row values. Displacement: action-chunk MSE from Start on task-8 observations, $\times 10^{-3}$. $^{\dagger}$Under the main experiments' precision; at full precision RL learns the new task (0.51) and keeps the prior one (0.69) (Section 4.4, Appendix G).

| | Task 64 (new) | | Task 8 (prior) | | |
|---|---|---|---|---|---|
| | success | $\Delta$ | success | $\Delta$ | displ. |
| Start | 0.35 | – | 0.69 | – | 0 |
| RL$^{\dagger}$ | 0.35 | +0.00 [−0.05, +0.05] | 0.73 | +0.04 [−0.03, +0.10] | 0.5 |
| Own | 0.41 | +0.06 [+0.01, +0.11] | 0.77 | +0.07 [+0.02, +0.12] | 8.1 |
| Demo+Own | 0.69 | +0.33 [+0.24, +0.42] | 0.70 | +0.01 [−0.07, +0.07] | 14.6 |
| Demo | 0.71 | +0.32 [+0.19, +0.45] | **0.00** | **−0.70 [−0.77, −0.63]** | 32.7 |

steps, executing the first 5 actions of each 10-step chunk before re-planning; because the RL conditions train under the stochastic sampler, we also evaluate the main checkpoints under it (Table 12). We report success with 95% bootstrap intervals over initial states, and differences from Start with a paired-by-state bootstrap interval and an exact McNemar test.

**Displacement.** The language-model account measures forgetting against the KL divergence between the fine-tuned and initial policies. Under evaluation the policy decodes deterministically and has no likelihood, so we use a likelihood-free stand-in: the mean squared error (MSE) between the 10-step action chunks a checkpoint and Start produce on the same 500 observations with the same initial noise. We measure it on prior-task observations, where the prior skill lives, and on bowl-stacking observations (its relation to the KL account is discussed in Section 5).

# 4 Results

## 4.1 Data source determines retention

Table 1 gives the final checkpoints. Human demonstrations learn the new task, raising new-task success from 0.40 to 0.71 ($\Delta$ =+0.32 [+0.19, +0.45], 3 runs), and erase the prior one: prior-task success falls from 0.70 to 0.00 in every run and at every evaluated checkpoint from step 200 on. The same demonstrations at their original 128 px, rather than re-rendered at the 256 px the policy sees at evaluation, learn less (0.49) and forget identically (0.00; Appendix C).

The policy's own rollouts keep the prior skill and learn the new task only modestly. Prior-task success ends at or above its starting value in all six Own runs (0.75–0.80; pooled $\Delta$ =+0.07 [+0.02, +0.12]; see Appendix I for the likely cause). New-task success rises from 0.35 to 0.41 pooled over 6 runs (2,400 paired episodes; $\Delta$ =+0.06 [+0.01, +0.11]); at full precision the same data lifts it to 0.55 while still keeping the prior skill (Appendix G). On two further new tasks the pattern repeats: demonstrations erase the prior skill and own rollouts keep it (Table 3).

## 4.2 Forgetting precedes learning

Figure 1 (left) follows Demo through training. After 100 gradient steps prior-task success has fallen to 0.14 and new-task success to 0.08. After 200 both are at 0.00, and at step 530 the new task is still near zero. Demonstrations first degrade the policy on both tasks and only then learn the new one, from about step 1060, while the prior task never returns. The 128 px run follows the same course (Appendix C). Early stopping on new-task success therefore cannot protect the prior skill: the skill is gone before new-task success has risen.

Under the policy's own rollouts pooled prior-task success stays between 0.73 and 0.78 at every evaluated step, while the new-task gain arrives in the second half of training (Figure 1, right).

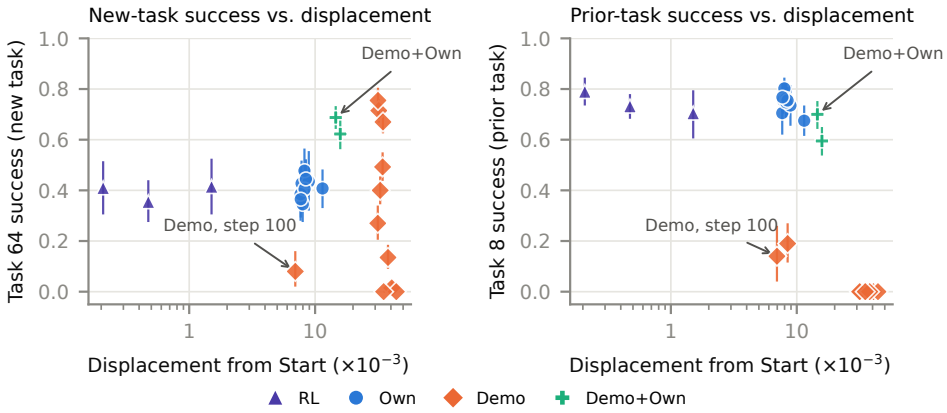



Figure 3: Success against displacement from Start (mean chunk MSE on 500 prior-task observations, log scale) for every bowl-stacking checkpoint with both measurements. **Left:** new-task success. **Right:** prior-task success. Neither outcome is a function of distance alone: at the own-rollout displacement, demonstration training has already lost most of the prior skill (step 100), and demonstrations with own replay move farther than own rollouts yet keep it. Bars: 95% bootstrap CIs over initial states.

Table 2: **Same distance, different forgetting.** Displacement from Start on task-8 observations ($\times 10^{-3}$); task-8 success with 95% CI over initial states (step 100: 50 episodes). Own pools six final runs (0.75–0.80 across runs). All checkpoints in Table 8.

| Checkpoint | Data source | Displacement | Task-8 success |
|---|---|---|---|
| Own, final (6 runs) | own rollouts | 8.1 | 0.77 [0.72, 0.80] |
| Demo, step 100 | human demos | 7.0 | 0.14 [0.04, 0.26] |

The failures point to a lost motor skill rather than a lost task: in the 128 px run's prior-task episodes, the policy still reaches the correct pot, but closes the gripper late and lifts the pot in only 40% of them. Replaying the policy's own prior-task rollouts (Demo+Own) does not prevent this early loss, with prior-task success at 0.12 at step 530, but reverses it over training, to 0.61 at step 1060 and 0.70 at step 2118 (Appendix D; Section 4.6).

### 4.3 Displacement is informative but incomplete

Figure 3 places every bowl-stacking checkpoint by its displacement from Start on prior-task observations (action MSE, $\times 10^{-3}$; all values in Table 9). The conditions form clusters: on-policy RL at or below 2, own-rollout SFT at 7.3–11.5, and final demonstration SFT at 31.6–44.2. Displacement separates the final outcomes on all three task pairs (Table 3): every evaluated final checkpoint displaced by more than 20 lost the prior skill, and every one displaced by less than 15 kept it. The only one in between, human-demonstration replay at 15.8, kept it partly (0.59).

Distance is not the whole story (Table 2). After 100 steps of training with human demonstrations the policy has moved about as far as the own-rollout runs ever do (7.0 vs. 8.1) yet has already lost most of the prior skill (0.14 vs. 0.77). Demo+Own, displaced nearly twice as far as Own (14.6), keeps the prior skill in full (0.70). RL's Razor finds that KL divergence predicts forgetting in language models where weight-space distance does not [18]. Our action-space measure reproduces that separation across final checkpoints, but two updates at the same distance can differ in whether the prior skill survives, which distance alone cannot predict.

Table 3: **Three task pairs.** Prior task is moka pots (task 8) throughout. Tasks 38 and 60: one run per condition, 200 paired episodes; details of their training sets in Table 4. Displacement: action-chunk MSE from Start on task-8 observations, $\times 10^{-3}$.

| New task | Condition | New-task success | Prior-task success | Displ. (t8 obs) |
|---|---|---|---|---|
| 64 (bowl stacking) | Start | 0.35 | 0.69 | 0 |
| | Own (6 runs) | 0.41 | 0.77 | 8.1 |
| | Demo (3 runs) | 0.71 | **0.00** | 32.7 |
| 38 (one moka pot) | Start | 0.42 | 0.70 | 0 |
| | Own | 0.40 | 0.69 | 11.0 |
| | Demo | 0.79 | **0.05** | 25.4 |
| 60 (bowl to tray) | Start | 0.45 | 0.70 | 0 |
| | Own | 0.49 | 0.71 | 6.2 |
| | Demo | 0.99 | **0.00** | 20.9 |

### 4.4 Does the objective matter? On-policy RL at full precision

The RL condition trains on the same rollouts as Own but with the GRPO objective. Under the main experiments' precision it barely moves the policy: five iterations leave the new task at 0.35 and the prior task at 0.73, and 15 iterations or lower exploration noise ($\sigma = 0.25$) give 0.41 against 0.40 for Start (Table 9). Its updates fell below bf16 resolution (Appendix B), so these runs cannot ablate the objective; a policy that does not move retains trivially.

We therefore repeat the RL, Own, and Demo conditions at full precision (Appendix G). On-policy GRPO learns the new task, 0.51 ($\Delta = +0.12$ $[+0.03, +0.21]$, $p = 0.018$), and keeps the prior skill (0.69). SFT on the same rollouts learns comparably (0.55, $+0.15$) and keeps it (0.79). Demonstrations learn more (0.90) and erase it (0.05, every initial state worse). On the same data, the two objectives learn and retain alike, which is what the language-model account predicts: the data, not the RL objective, carries the retention [18, 19].

This is one task pair and one seed per condition. On it, the two objectives differed in how far they moved the policy (RL 1.11 vs. SFT 4.86), but the difference showed in neither learning nor retention; as in Section 4.3, distance moved did not decide the outcome, and the data source did.

### 4.5 The demonstrations are far off-policy

We measured how far each training set's actions lie from what Start would do at the same observations (Appendix H). Demonstration actions are roughly an order of magnitude farther from the policy's own than its successful rollouts are: mean chunk MSE 148.8 vs. 30.1 ($\times 10^{-3}$), of which resampling noise accounts for 24.4 and 15.8 (Table 11). The gap is spread across all 50 demonstrations and concentrated in the middle of each episode, where the object is grasped and carried. Over half of it is in the gripper channel, as a timing offset: the demonstrator closes earlier and holds longer than the policy. Even with the gripper excluded, the demonstrations' arm motions are eightfold farther from the policy's than its rollouts are.

### 4.6 Replication, replay, and variants

Table 3 repeats the comparison on two more new tasks against the same prior task: *one moka pot* (task 38), which shares the pots and stove, and *bowl to tray* (task 60), which is unrelated. The pattern holds on both: demonstrations learn the new task and nearly erase the prior skill, own rollouts keep the prior skill and learn neither new task, and own rollouts move the policy less in every pair. Given that full precision lifted own-rollout learning on bowl stacking (Section 4.4), rerunning these pairs at full precision is a natural next step.

On bowl stacking, replaying the policy's own prior-task rollouts into the demonstrations (Demo+Own, about a third of the frames) keeps the prior skill (0.70) at the demonstrations' level of

learning, while the same share of human prior-task demonstrations keeps it only partly (0.59) and learns less, at a similar displacement. Replaying the policy's own data protects better than replaying human data for the same task.

## 5 Discussion and Limitations

**Why demonstrations erase the skill.** The demonstrations' off-policy gap is largest in gripper timing (Section 4.5), and the skill they erase fails at exactly this point: the policy still reaches the correct pot but closes late (Section 4.2). Where the update lands in weight space does not separate the conditions: Own and Demo have the same per-block profile and the same precision-frozen fraction, and their overlap with the starting weights' principal directions depends on training length rather than data source (Appendix B). We hypothesize that fitting an off-policy action schedule, above all for the gripper, overwrites the schedule the prior skill depends on. The intervention that would test this, replacing one action channel of the demonstrations with the policy's own, is future work.

**Own data protects, and has a ceiling.** Filtered behavior cloning on a robot's own successful rollouts of a new task [36, 37], with no prior-task data, kept the prior skill in every run on all three new tasks and at both precisions, while demonstrations erased it on all three and, on the task we tracked through training, before teaching anything. What the same data can teach is bounded by what the policy already sometimes does. At the main experiments' precision it learned one of three new tasks, modestly (+0.06); at full precision the same data lifts bowl stacking by $+0.15$ (Appendix G), still well short of demonstrations ($+0.50$). Part of the ceiling was precision; the rest is the familiar bound of self-imitation.

**Replay and its limits.** Replay protects only what is replayed [30, 15], whereas own-rollout training on the new task protected the prior skill without any prior-task data. If that holds for many skills at once, which we have not tested, its cost would not grow with the repertoire.

**What to collect next.** Own rollouts need a policy that sometimes succeeds; all three new tasks start at 0.35–0.45. Where it does not, human corrections at failure states [33, 34, 38, 39] add off-policy data only where it is needed, keeping the rest near the policy. We expect such data to learn faster than own rollouts and forget less than demonstrations, which we have not tested.

**Relation to the language-model account.** Our results agree with the account's main point, that on-policy data rather than the objective is what protects prior capabilities [18, 19]: SFT and RL on the same rollouts retained alike at full precision. They qualify it in one way. The account measures forgetting by divergence from the initial policy; our action-space distance separates retained from forgotten final checkpoints the same way, but the data source acts beyond it: demonstrations at step 100 and final own-rollout checkpoints sit at a similar displacement with opposite outcomes (Section 4.3). A property of the data measured before training, how far its actions lie from the policy's own (Section 4.5), may be the better predictor. We leave this as an open question.

**Limitations.** Our experiments are limited to simulation, one model family, and a single prior skill improved by RL, tested against three new tasks. Because the prior task appears in the released checkpoint's training data, retention may be easier than for a skill the model never saw. The one-moka-pot and bowl-to-tray comparisons are single runs, and evaluations use 200 to 400 paired episodes per final checkpoint. All main-text results trained in bfloat16 rather than at full precision, which left most of the action expert's transformer weights unchanged in every condition (Appendix B); they characterize updates carried mainly by the adaptive-norm and projection parameters. The full-precision replication covers one task pair with one seed per condition. Future work includes replication on a real arm and an RL exploration strategy that finds successes the policy does not already produce.

## References


[1] Physical Intelligence, K. Black, N. Brown, J. Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[2] Physical Intelligence. $\pi_{0.7}$: a steerable model with emergent capabilities. https://www.pi.website/blog/pi07, 2026.

[3] Gemini Robotics Team. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer. *arXiv preprint arXiv:2510.03342*, 2025.

[4] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024. arXiv:2310.08864.

[5] Generalist AI. GEN-1: Scaling embodied foundation models to mastery. https://generalistai.com/blog/gen-1, Apr. 2026.

[6] Generalist AI. GEN-1.5: Embodied foundation models are one-shot learners. https://generalistai.com/blog/gen-1.5, Aug. 2026.

[7] Dyna Robotics. Dynamism v1 (DYNA-1) model: A breakthrough in performance and production-ready embodied AI. https://www.dyna.co/research/dyna-1, 2025.

[8] Covariant. Covariant introduces RFM-1 to give robots the human-like ability to reason. https://covariant.ai/covariant-introduces-rfm-1-to-give-robots-the-human-like-ability-to-reason/, 2024. March 11, 2024.

[9] 1X Technologies. NEO: The home robot. https://www.1x.tech/discover/neo-home-robot, 2025. Announced October 28, 2025; consumer deliveries planned for 2026.

[10] Sunday Robotics. Sunday launches Memo, the robot that actually learns your home. https://www.globenewswire.com/news-release/2025/11/19/3191274/0/en/Sunday-Launches-Memo-the-Robot-That-Actually-Learns-Your-Home.html, 2025. Press release, November 19, 2025.

[11] T. Lesort, V. Lomonaco, A. Stoian, D. Maltoni, D. Filliat, and N. Díaz-Rodríguez. Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, 58:52–68, 2020.

[12] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. arXiv:2306.03310.

[13] M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

[14] R. M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.

[15] P. Garg, S. Xing, P. Yaugand, S. Gupta, and D. Hoiem. Fine-tuning VLAs with self-demonstrated generative control for multi-task manipulation. *arXiv preprint arXiv:2608.19490*, 2026.

[16] J. Zhu, Y. Hong, X. Sun, Z. Xu, Q. He, H. Chen, Z. Wang, M. Yuan, W. Zeng, and J. Chen. Can vision-language-action models learn from real-world data continually without forgetting? *arXiv preprint arXiv:2605.26820*, 2026.

[17] J. Hu, J. Shim, C. Tang, Y. Sung, B. Liu, P. Stone, and R. Martín-Martín. Simple recipe works: Vision-language-action models are natural continual learners with reinforcement learning. *Reinforcement Learning Journal*, 2026. arXiv:2603.11653.

[18] I. Shenfeld, J. Pari, and P. Agrawal. RL's razor: Why online reinforcement learning forgets less. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2509.04259.

[19] H. Chen, N. Razin, K. Narasimhan, and D. Chen. Retaining by doing: The role of on-policy data in mitigating forgetting. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2510.18874.

[20] W. Xiao, H. Lin, A. Peng, H. Xue, T. He, Y. Xie, F. Hu, J. Wu, Z. Luo, L. Fan, G. Shi, and Y. Zhu. Self-improving vision-language-action models with data generation via residual RL. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2511.00091.

[21] T. Zhang, S. Zhai, H. Zhang, F. Huang, and Q. Zhang. Preserving foundational capabilities in flow-matching VLAs through conservative SFT. *arXiv preprint arXiv:2605.08879*, 2026.

[22] H. Liu, C. Kim, B. Liu, M. Liu, and Y. Zhu. Pretrained vision-language-action models are surprisingly resistant to forgetting in continual learning. *arXiv preprint arXiv:2603.03818*, 2026.

[23] U. B. Karli and T. Fitzgerald. RECALL: Recovery experience collection for active lifelong learning in vision-language-action models. *arXiv preprint arXiv:2606.23617*, 2026.

[24] R. M. S. Khan, Z. Liu, Z. Tan, C. Fleming, and T. Chen. TMS: Trajectory-mixed supervision for reward-free, on-policy SFT. *arXiv preprint arXiv:2602.03073*, 2026.

[25] Z. Yang, T. Pang, H. Feng, H. Wang, W. Chen, M. Zhu, and Q. Liu. Self-distillation bridges distribution gap in language model fine-tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2402.13669.

[26] I. Shenfeld, M. Damani, J. Hübotter, and P. Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.

[27] J. Hübotter, F. Lübeck, L. Behric, A. Baumann, M. Bagatella, D. Marta, I. Hakimi, I. Shenfeld, T. Kleine Buening, C. Guestrin, and A. Krause. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.

[28] K. Lu and Thinking Machines Lab. On-policy distillation. https://thinkingmachines.ai/blog/on-policy-distillation/, 2025.

[29] Y. Chen, T. Zhu, M. Zhang, X. Chen, J. Huang, D. Xu, and Y. Wang. On-policy replay for continual supervised fine-tuning. *arXiv preprint arXiv:2605.29495*, 2026.

[30] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1811.11682.

[31] A. Robins. Catastrophic forgetting, rehearsal and pseudorehearsal. *Connection Science*, 7(2): 123–146, 1995.

[32] H. Shin, J. K. Lee, J. Kim, and J. Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. arXiv:1705.08690.

[33] Z. Hu, R. Wu, N. Enock, J. Li, R. Kadakia, Z. Erickson, and A. Kumar. RaC: Robot learning for long-horizon tasks by scaling recovery and correction. *arXiv preprint arXiv:2509.07953*, 2025.

[34] Physical Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, et al. $\pi^*_{0.6}$: a VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

[35] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin, H. Zang, X. Li, Q. Zhang, Z. Yu, G. Fan, T. Huang, Y. Wang, and C. Yu. $\pi_{\mathrm{rl}}$: Online RL fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*, 2025.

[36] J. Oh, Y. Guo, S. Singh, and H. Lee. Self-imitation learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 3878–3887, 2018.

[37] C. Gulcehre, T. L. Paine, S. Srinivasan, K. Konyushkova, L. Weerts, A. Sharma, A. Siddhant, A. Ahern, M. Wang, C. Gu, W. Macherey, A. Doucet, O. Firat, and N. de Freitas. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

[38] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu. Robot learning on the job: Human-in-the-loop autonomy and learning during deployment. In *Robotics: Science and Systems (RSS)*, 2023. arXiv:2211.08416.

[39] J. Luo, C. Xu, J. Wu, and S. Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105):eads5033, 2025. arXiv:2410.21845.

[40] M. T. Yardımcı and Y. E. Çoğurcu. Uncertainty-gated exploration noise suppresses task collapse in online RL fine-tuning of a flow-matching vision-language-action policy. *arXiv preprint arXiv:2609.28838*, 2026.

# A Run index

Runs are named Condition/variant throughout the appendix (Own/rl-rollouts-s1, Demo/256px, Demo+Own/own-replay, RL/15it); Table 4 gives each run's log id for matching against the training logs. Suffixes: -s1/-s2 seeds, @step a checkpoint before the end of training, (SDE) evaluation under the training sampler.

Table 4: **Runs, their log ids, and training sets.** Log ids match the released checkpoints and run logs. All runs start from Start (step20); supervised runs share the objective, optimizer and batch size, and train for 2118 steps except the fine-checkpoint runs (530) and Demo/task38-128px (evaluated at step 2118 of a 4236-step schedule). Episode and frame counts are the training sets after success filtering, from the training launch logs. Own/start-rollouts and Own/rl-rollouts use two collection passes, Start's own and RL/5it's training rollouts, which sample nearly the same distribution because RL barely moves the policy at the main experiments' precision (Section 4.4). Seeds carry a suffix (-s1, -s2); checkpoints before the end of training carry @step.

| Run | Log id | Training data | Episodes | Frames |
|---|---|---|---|---|
| Start | step20 | released checkpoint + 20 GRPO iterations on task 8 | 1280 rollouts | |
| RL/5it | A | on-policy GRPO on task 64, 5 iterations | 320 rollouts | |
| RL/$\sigma$0.25 | A_s025 | as RL/5it, $\sigma = 0.25$ | 320 rollouts | |
| RL/15it | A_long | as RL/5it, 15 iterations | 960 rollouts | |
| Own/start-rollouts | B | Start's successful task-64 rollouts (of 320) | 154 | 51,294 |
| Own/rl-rollouts | B_RL | RL/5it's successful task-64 rollouts (of 320) | 143 | 45,545 |
| Own/rl-rollouts-505ep | B_RL_long | successes of RL/5it + RL/15it rollouts | 505 | 154,614 |
| Own/round2 | loop1 | successes of Own/rl-rollouts' rollouts (of 320) | 155 | 50,280 |
| Own/lr4x | B_hiLR | as Own/start-rollouts, learning rate $2 \times 10^{-5}$ | 154 | 51,294 |
| Own/lr2x | B_RL_lr1e5 | as Own/rl-rollouts, learning rate $1 \times 10^{-5}$ | 143 | 45,545 |
| Demo+Own/own-replay | D | Demo/256px's demonstrations + 17 Start task-8 successes | 67 | 18,549 |
| Demo+Own/demo-replay | D_demos | Demo/256px's demonstrations + 17 human task-8 demonstrations | 67 | 18,804 |
| Demo/128px | C | human demonstrations of task 64 (128 px) | 50 | 11,734 |
| Demo/256px | C256 | as Demo/128px, rendered at 256 px | 50 | 11,734 |
| Demo/lr0.4x, lr0.2x | C_lr2e6, C_lr1e6 | as Demo/128px, learning rates $2, 1 \times 10^{-6}$ | 50 | 11,734 |
| Demo/128px-fine, 256px-fine | C_fine, C256_fine | as Demo/128px, Demo/256px; first 530 steps, checkpoint every 100 | 50 | 11,734 |
| Own/task38 | B_RL_38 | successful task-38 rollouts of RL on task 38 (of 320) | 258 | 102,603 |
| Demo/task38-128px | C38 | human demonstrations of task 38 (128 px), step 2118 of 4236 | 50 | 10,588 |
| Own/task60 | Own_60 | Start's successful task-60 rollouts (of 320) | 163 | 20,045 |
| Demo/task60 | Demo_60 | human demonstrations of task 60 (256 px) | 50 | 6,176 |

# B Training budget

Every supervised run uses AdamW at a learning rate of $5 \times 10^{-6}$ and a global batch of 64; the learning-rate controls scale that rate. The SFT schedule was set from the demonstration set: with 50 demonstrations (11,734 frames) and a global batch of 64, the episode-pass rule used for $\pi_{0.5}$ fine-tuning gives 4236 gradient steps. We capped every supervised condition at half of that, 2118 steps, so that the larger own-rollout sets (up to 154,614 frames) would finish within the compute available, and applied the same cap to the demonstration conditions to keep the budget identical across conditions. Larger training sets therefore see fewer passes over their data, not more steps. The fine-checkpoint runs train for 530 steps, and Demo/task38-128px is evaluated at step 2118 of a 4236-step schedule. Checkpoints were saved every 530 steps in every run, and every 100 steps in the fine-checkpoint demonstration runs.

**Precision.** All runs train the action expert in bfloat16 without fp32 master weights: parameters are loaded in bf16 (the loader keeps the adaptive-RMSNorm maps, the action and time projections, and the layer norms in fp32), FSDP mixed precision is disabled, and AdamW updates each parameter in its stored dtype. We refer to training with fp32 master weights as full precision. At a learning rate of $5 \times 10^{-6}$ the per-step update is below bf16's half-spacing for any entry with $|w| \geq 2^{-9}$, so such entries cannot move. After training, 97.6% of the expert's attention and MLP entries are bit-identical to Start in every condition, no entry with $|w| \geq 2 \times 10^{-3}$ changed, and about 98% of the update energy lies in the fp32 adaptive-norm and projection matrices (Table 5). This regime is shared by all conditions and does not bias the comparison between data sources, but it bounds what any condition could learn and is the reason the RL conditions barely moved (Section 4.4; see also Yardımcı and Çoğurcu [40]). Appendix G repeats three conditions at full precision.

Table 5: **Where the bf16 update lives.** $\|\Delta W\|_F$ per matrix family and the fraction of entries bit-identical to Start (weighted by entry count). attn: $q, k, v, o$ (bf16); mlp: gate, up, down (bf16); adaRMS: the per-layer adaptive-RMSNorm dense maps (fp32); proj: action in/out projections, time MLP and final-norm dense (fp32).

| | $\|\Delta W\|_F$ | | | | fraction of entries unchanged | | | |
|---|---|---|---|---|---|---|---|---|
| checkpoint | attn | mlp | adaRMS | proj | attn | mlp | adaRMS | proj |
| Own-final (B_RL) | 0.766 | 1.14 | 6.78 | 1.41 | 0.958 | 0.957 | $< 10^{-4}$ | 0.198 |
| Demo-final (C256) | 1.05 | 1.48 | 9.03 | 1.97 | 0.957 | 0.955 | $< 10^{-4}$ | 0.198 |
| RL/15it | 0.522 | 0.897 | 4.98 | 0.722 | 0.956 | 0.954 | $< 10^{-4}$ | 0.199 |
| Demo@100 | 0.366 | 0.567 | 2.29 | 0.506 | 0.957 | 0.955 | $< 10^{-4}$ | 0.198 |

# C Demo (128 px) by training step

Table 6: **Demo/128px by training step.** Deterministic protocol; step 0 is Start; steps 100 and 200 come from the fine-checkpoint run (Demo/128px-fine). 128 px demonstrations learn less than the 256 px run with the same seed (0.492 against 0.715 on the new task) and forget identically (0.000 on the prior task). Success with 95% bootstrap CI over initial states.

| Step | Task 64 (new) | Task 8 (prior) |
|---|---|---|
| 0 | 0.355 [0.27, 0.44] | 0.695 [0.64, 0.75] |
| 100 | – | 0.190 [0.12, 0.27] |
| 200 | – | 0.000 [0.00, 0.00] |
| 530 | 0.015 [0.00, 0.04] | 0.000 [0.00, 0.00] |
| 1060 | 0.270 [0.20, 0.34] | 0.000 [0.00, 0.00] |
| 1590 | 0.400 [0.34, 0.46] | 0.000 [0.00, 0.00] |
| 2118 | 0.492 [0.44, 0.55] | 0.000 [0.00, 0.00] |

# D Demo+Own by training step

Table 7: **Demo+Own/own-replay by training step.** Demo/256px's demonstrations plus 17 of Start's own prior-task successes; deterministic protocol; step 0 is Start. The prior skill falls at step 530 and recovers over training; the new task was evaluated only at the end. Success with 95% bootstrap CI over initial states.

| Step | Task 64 (new) | Task 8 (prior) |
|---|---|---|
| 0 | 0.355 [0.27, 0.44] | 0.695 [0.64, 0.75] |
| 530 | – | 0.115 [0.07, 0.15] |
| 1060 | – | 0.610 [0.55, 0.68] |
| 2118 | 0.688 [0.64, 0.73] | 0.700 [0.64, 0.75] |

# E Learning-rate controls and matched displacement

**Learning-rate controls.** Changing the learning rate does not change where either data source ends up. Own-rollout SFT at two and four times the default rate stays in the own-rollout cluster (8.9 and 7.3) and, at twice the rate, still keeps the prior skill. Demonstration SFT at $0.4\times$ and $0.2\times$ the rate ends as far from Start as at the default rate, still loses the prior skill entirely, and learns less of the new task, none at all at $0.2\times$: at low rates, training never gets past the phase in which demonstrations degrade both tasks (Section 4.2).

Table 8: **Extended matched-displacement table.** Every checkpoint with a task-8 displacement, grouped by condition and sorted by displacement, including the learning-rate controls and the task-38 and task-60 runs. Displacement is action-chunk MSE from Start on task-8 observations, $\times 10^{-3}$.

| Checkpoint | Data source | Displacement | Task-8 success |
|---|---|---|---|
| *RL* | | | |
| RL/$\sigma$0.25 | RL | 0.2 | 0.79 [0.73, 0.84] |
| RL/5it | RL | 0.5 | 0.73 [0.68, 0.78] |
| RL/15it@it5 | RL | 0.6 | – |
| RL/15it@it15 | RL | 1.5 | 0.70 [0.60, 0.80] |
| RL/15it@it10 | RL | 1.7 | – |
| *Own* | | | |
| Own/lr4x | Own | 7.3 | – |
| Own/rl-rollouts-505ep | Own | 7.7 | 0.70 [0.62, 0.78] |
| Own/start-rollouts-s2 | Own | 7.7 | 0.77 [0.71, 0.82] |
| Own/rl-rollouts | Own | 7.8 | 0.76 [0.70, 0.80] |
| Own/start-rollouts | Own | 8.0 | 0.80 [0.76, 0.84] |
| Own/rl-rollouts-s2 | Own | 8.2 | 0.75 [0.69, 0.81] |
| Own/rl-rollouts-s1 | Own | 8.2 | 0.76 [0.69, 0.82] |
| Own/start-rollouts-s1 | Own | 8.5 | 0.76 [0.70, 0.81] |
| Own/lr2x | Own | 8.9 | 0.73 [0.66, 0.81] |
| Own/round2 | Own | 11.5 | 0.68 [0.61, 0.73] |
| *Demo* | | | |
| Demo/256px-fine@100 | Demo | 7.0 | 0.14 [0.04, 0.26] |
| Demo/128px-fine@100 | Demo | 8.5 | 0.19 [0.12, 0.27] |
| Demo/128px@1060 | Demo | 31.5 | 0.00 [0.00, 0.00] |
| Demo/256px-s2 | Demo | 31.6 | 0.00 [0.00, 0.00] |
| Demo/256px | Demo | 32.1 | 0.00 [0.00, 0.00] |
| Demo/128px@1590 | Demo | 32.9 | 0.00 [0.00, 0.00] |
| Demo/128px-s2 | Demo | 34.3 | – |
| Demo/128px | Demo | 34.4 | 0.00 [0.00, 0.00] |
| Demo/256px-s1 | Demo | 34.6 | 0.00 [0.00, 0.00] |
| Demo/256px-fine@400 | Demo | 34.7 | – |
| Demo/256px-fine@500 | Demo | 35.0 | – |
| Demo/256px-fine@200 | Demo | 35.0 | 0.00 [0.00, 0.00] |
| Demo/256px-fine@300 | Demo | 35.1 | – |
| Demo/256px-fine@530 | Demo | 36.2 | – |
| Demo/lr0.4x | Demo | 37.9 | 0.00 [0.00, 0.00] |
| Demo/128px-s1 | Demo | 38.3 | – |
| Demo/128px@530 | Demo | 40.6 | 0.00 [0.00, 0.00] |
| Demo/128px-fine@500 | Demo | 41.5 | – |
| Demo/128px-fine@530 | Demo | 41.5 | – |
| Demo/128px-fine@400 | Demo | 42.0 | – |
| Demo/lr0.2x | Demo | 44.2 | 0.00 [0.00, 0.00] |
| Demo/128px-fine@300 | Demo | 46.9 | – |
| Demo/128px-fine@200 | Demo | 47.9 | 0.00 [0.00, 0.00] |
| *Demo+Own* | | | |
| Demo+Own/own-replay | Demo+Own | 14.6 | 0.70 [0.64, 0.75] |
| Demo+Own/demo-replay | Demo+Own | 15.8 | 0.59 [0.54, 0.65] |
| *Other task pairs* | | | |
| Own/task60 | Own | 6.2 | 0.71 [0.61, 0.80] |
| Own/task38 | Own | 11.0 | 0.69 [0.58, 0.79] |
| Demo/task60 | Demo | 20.9 | 0.00 [0.00, 0.00] |
| Demo/task38-128px | Demo | 25.4 | 0.05 [0.01, 0.09] |

# F All evaluated checkpoints

Every bowl-stacking run, both tasks, with the protocol and episode count of each evaluation; the other two task pairs are in Table 3.

Table 9: **Every bowl-stacking checkpoint with an evaluation or a distance measurement;** the task-38 and task-60 runs are in Table 3. Table 1's Own row pools the final checkpoints of Own/start-rollouts, Own/start-rollouts-s1, Own/start-rollouts-s2, Own/rl-rollouts, Own/rl-rollouts-s1, Own/rl-rollouts-s2. Success with 95% bootstrap CI over initial states; $\Delta$ vs. Start (paired-bootstrap CI); $p$ = exact McNemar over paired episodes; distance = mean chunk MSE to Start on 500 fixed observations of that task, $\times 10^{-3}$. SDE rows use the training sampler ($\sigma = 0.5$, 2 draws) and are compared with Start under the same sampler. Start (quick) re-runs Start on the 50-episode subset; its $\Delta$ against Start's original episodes estimates run-to-run evaluation noise.

| Checkpoint | Protocol | Task 64 (new) | | | Task 8 (prior) | | | Distance | |
|---|---|---|---|---|---|---|---|---|---|
| | | success | $\Delta$ | $p$ | success | $\Delta$ | $p$ | t64 | t8 |
| *Start* | | | | | | | | | |
| Start, pre-RL | ODE 200 | 0.380 [0.27, 0.51] | −0.015 [−0.09, +0.06] | 0.807 | 0.545 [0.43, 0.66] | −0.155 [−0.28, −0.04] | 0.002 | – | – |
| Start | ODE 400 | 0.355 [0.27, 0.44] | – | – | 0.695 [0.64, 0.75] | – | – | – | – |
| Start (quick) | ODE 50 | 0.380 [0.24, 0.54] | −0.040 [−0.18, +0.10] | 0.815 | – | – | – | – | – |
| Start (SDE) | SDE 50 | 0.360 [0.22, 0.50] | – | – | 0.880 [0.80, 0.96] | – | – | – | – |
| *RL* | | | | | | | | | |
| RL/5it@iter1 | – | – | – | – | – | – | – | 1.03 | – |
| RL/5it@iter2 | – | – | – | – | – | – | – | 1.04 | – |
| RL/5it@iter3 | – | – | – | – | – | – | – | 1.21 | – |
| RL/5it@iter4 | – | – | – | – | – | – | – | 1.11 | – |
| RL/5it | ODE 400 | 0.355 [0.28, 0.44] | +0.000 [−0.05, +0.05] | 1.000 | 0.733 [0.68, 0.78] | +0.038 [−0.03, +0.10] | 0.258 | 0.77 | 0.47 |
| RL/5it (SDE) | SDE 50 | 0.360 [0.22, 0.50] | +0.000 [−0.16, +0.16] | 1.000 | 0.720 [0.60, 0.84] | −0.160 [−0.32, +0.00] | 0.077 | – | – |
| RL/$\sigma$0.25 | ODE 200 | 0.410 [0.30, 0.52] | +0.015 [−0.07, +0.10] | 0.818 | 0.790 [0.73, 0.84] | +0.090 [+0.01, +0.17] | 0.060 | – | 0.21 |
| RL/15it@it5 | – | – | – | – | – | – | – | – | 0.58 |
| RL/15it@it10 | – | – | – | – | – | – | – | – | 1.75 |
| RL/15it@it15 | ODE 200 | 0.415 [0.30, 0.53] | +0.020 [−0.06, +0.10] | 0.704 | 0.705 [0.60, 0.80] | +0.005 [−0.14, +0.14] | 1.000 | – | 1.50 |
| *Own* | | | | | | | | | |
| Own/start-rollouts | ODE 400 | 0.345 [0.28, 0.42] | −0.010 [−0.07, +0.05] | 0.814 | 0.802 [0.76, 0.84] | +0.108 [+0.04, +0.17] | <0.001 | 24.47 | 7.97 |
| Own/start-rollouts (SDE) | SDE 50 | 0.480 [0.32, 0.64] | +0.120 [−0.02, +0.26] | 0.210 | 0.840 [0.74, 0.92] | −0.040 [−0.14, +0.06] | 0.774 | – | – |
| Own/start-rollouts-s1 | ODE 400 | 0.445 [0.37, 0.52] | +0.090 [+0.03, +0.15] | 0.005 | 0.755 [0.70, 0.81] | +0.060 [+0.00, +0.12] | 0.052 | 23.96 | 8.45 |
| Own/start-rollouts-s2 | ODE 400 | 0.365 [0.29, 0.44] | +0.010 [−0.04, +0.06] | 0.789 | 0.767 [0.71, 0.82] | +0.073 [−0.00, +0.14] | 0.022 | 22.11 | 7.69 |
| Own/lr4x | – | – | – | – | – | – | – | – | 7.30 |
| Own/rl-rollouts@530 | ODE 50 | 0.360 [0.22, 0.50] | −0.060 [−0.22, +0.10] | 0.664 | 0.720 [0.60, 0.84] | +0.000 [−0.16, +0.16] | 1.000 | – | – |
| Own/rl-rollouts@1060 | ODE 50 | 0.320 [0.20, 0.44] | −0.100 [−0.28, +0.08] | 0.359 | 0.760 [0.62, 0.88] | +0.040 [−0.10, +0.18] | 0.815 | – | – |
| Own/rl-rollouts@1590 | ODE 50 | 0.560 [0.42, 0.70] | +0.140 [−0.02, +0.30] | 0.189 | 0.780 [0.66, 0.88] | +0.060 [−0.08, +0.20] | 0.629 | – | – |
| Own/rl-rollouts | ODE 400 | 0.427 [0.34, 0.52] | +0.073 [+0.01, +0.13] | 0.014 | 0.755 [0.70, 0.80] | +0.060 [−0.01, +0.13] | 0.072 | 23.51 | 7.81 |
| Own/rl-rollouts-s1@530 | ODE 50 | 0.320 [0.16, 0.48] | −0.100 [−0.22, +0.02] | 0.267 | 0.860 [0.74, 0.96] | +0.140 [+0.02, +0.26] | 0.143 | – | – |
| Own/rl-rollouts-s1@1060 | ODE 50 | 0.360 [0.22, 0.50] | −0.060 [−0.20, +0.08] | 0.629 | 0.740 [0.60, 0.88] | +0.020 [−0.14, +0.18] | 1.000 | – | – |
| Own/rl-rollouts-s1@1590 | ODE 50 | 0.480 [0.34, 0.64] | +0.060 [−0.12, +0.24] | 0.629 | 0.760 [0.64, 0.88] | +0.040 [−0.14, +0.20] | 0.824 | – | – |
| Own/rl-rollouts-s1 | ODE 400 | 0.405 [0.33, 0.48] | +0.050 [−0.01, +0.11] | 0.105 | 0.762 [0.69, 0.82] | +0.068 [+0.01, +0.13] | 0.029 | 23.19 | 8.25 |
| Own/rl-rollouts-s2@530 | ODE 50 | 0.200 [0.10, 0.32] | −0.220 [−0.34, −0.10] | 0.007 | 0.760 [0.62, 0.88] | +0.040 [−0.10, +0.16] | 0.804 | – | – |
| Own/rl-rollouts-s2@1060 | ODE 50 | 0.500 [0.36, 0.64] | +0.080 [−0.10, +0.26] | 0.523 | 0.700 [0.56, 0.82] | −0.020 [−0.18, +0.12] | 1.000 | – | – |
| Own/rl-rollouts-s2@1590 | ODE 50 | 0.500 [0.32, 0.68] | +0.080 [−0.08, +0.24] | 0.454 | 0.760 [0.62, 0.88] | +0.040 [−0.12, +0.20] | 0.824 | – | – |
| Own/rl-rollouts-s2 | ODE 400 | 0.477 [0.39, 0.56] | +0.122 [+0.05, +0.20] | <0.001 | 0.748 [0.69, 0.81] | +0.053 [−0.01, +0.11] | 0.085 | 23.22 | 8.23 |
| Own/lr2x | ODE 200 | 0.435 [0.32, 0.56] | +0.040 [−0.05, +0.14] | 0.366 | 0.735 [0.66, 0.81] | +0.035 [−0.05, +0.12] | 0.500 | 21.82 | 8.91 |
| Own/rl-rollouts-505ep | ODE 200 | 0.385 [0.28, 0.49] | −0.010 [−0.10, +0.08] | 0.899 | 0.705 [0.62, 0.78] | +0.005 [−0.09, +0.10] | 1.000 | 22.84 | 7.67 |
| Own/round2 | ODE 400 | 0.407 [0.33, 0.48] | +0.052 [−0.01, +0.12] | 0.080 | 0.675 [0.61, 0.73] | −0.020 [−0.10, +0.06] | 0.600 | 27.77 | 11.45 |
| *Demo* | | | | | | | | | |
| Demo/128px@530 | ODE 200 | 0.015 [0.00, 0.04] | −0.380 [−0.51, −0.26] | <0.001 | 0.000 [0.00, 0.00] | −0.700 [−0.77, −0.63] | <0.001 | – | 40.61 |
| Demo/128px@1060 | ODE 200 | 0.270 [0.20, 0.34] | −0.125 [−0.27, +0.01] | 0.014 | 0.000 [0.00, 0.00] | −0.700 [−0.77, −0.63] | <0.001 | – | 31.47 |
| Demo/128px@1590 | ODE 200 | 0.400 [0.34, 0.46] | +0.005 [−0.14, +0.14] | 1.000 | 0.000 [0.00, 0.00] | −0.700 [−0.77, −0.63] | <0.001 | – | 32.88 |
| Demo/128px | ODE 400 | 0.492 [0.44, 0.55] | +0.138 [+0.04, +0.23] | <0.001 | 0.000 [0.00, 0.00] | −0.695 [−0.75, −0.64] | <0.001 | – | 34.37 |
| Demo/128px (SDE) | SDE 50 | 0.480 [0.34, 0.62] | +0.120 [−0.06, +0.30] | 0.327 | 0.000 [0.00, 0.00] | −0.880 [−0.96, −0.80] | <0.001 | – | – |
| Demo/128px-s1@530 | ODE 50 | 0.020 [0.00, 0.06] | −0.400 [−0.56, −0.24] | <0.001 | 0.000 [0.00, 0.00] | −0.720 [−0.82, −0.62] | <0.001 | – | – |
| Demo/128px-s1@1060 | ODE 50 | 0.300 [0.16, 0.46] | −0.120 [−0.30, +0.06] | 0.263 | 0.000 [0.00, 0.00] | −0.720 [−0.82, −0.62] | <0.001 | – | – |
| Demo/128px-s1@1590 | ODE 50 | 0.520 [0.40, 0.66] | +0.100 [−0.10, +0.28] | 0.442 | 0.000 [0.00, 0.00] | −0.720 [−0.82, −0.62] | <0.001 | – | – |
| Demo/128px-s1 | – | – | – | – | – | – | – | 61.27 | 38.31 |
| Demo/128px-s2@530 | ODE 50 | 0.020 [0.00, 0.06] | −0.400 [−0.54, −0.26] | <0.001 | 0.000 [0.00, 0.00] | −0.720 [−0.82, −0.62] | <0.001 | – | – |
| Demo/128px-s2@1060 | ODE 50 | 0.180 [0.08, 0.28] | −0.240 [−0.44, −0.04] | 0.029 | 0.000 [0.00, 0.00] | −0.720 [−0.82, −0.62] | <0.001 | – | – |
| Demo/128px-s2@1590 | ODE 50 | 0.280 [0.16, 0.40] | −0.140 [−0.34, +0.06] | 0.189 | 0.000 [0.00, 0.00] | −0.720 [−0.82, −0.62] | <0.001 | – | – |
| Demo/128px-s2 | – | – | – | – | – | – | – | 61.69 | 34.29 |
| Demo/lr0.4x | ODE 200 | 0.135 [0.09, 0.18] | −0.260 [−0.39, −0.14] | <0.001 | 0.000 [0.00, 0.00] | −0.700 [−0.77, −0.63] | <0.001 | 61.67 | 37.93 |
| Demo/lr0.2x | ODE 200 | 0.000 [0.00, 0.00] | −0.395 [−0.52, −0.28] | <0.001 | 0.000 [0.00, 0.00] | −0.700 [−0.77, −0.63] | <0.001 | 64.38 | 44.16 |
| Demo/256px@530 | ODE 200 | 0.015 [0.00, 0.03] | −0.380 [−0.51, −0.26] | <0.001 | 0.000 [0.00, 0.00] | −0.700 [−0.77, −0.63] | <0.001 | – | – |
| Demo/256px@1060 | ODE 50 | 0.320 [0.20, 0.44] | −0.100 [−0.28, +0.08] | 0.383 | 0.000 [0.00, 0.00] | −0.720 [−0.82, −0.62] | <0.001 | – | – |
| Demo/256px@1590 | ODE 50 | 0.740 [0.62, 0.84] | +0.320 [+0.10, +0.54] | 0.004 | 0.000 [0.00, 0.00] | −0.720 [−0.82, −0.62] | <0.001 | – | – |
| Demo/256px | ODE 400 | 0.715 [0.67, 0.76] | +0.360 [+0.26, +0.46] | <0.001 | 0.000 [0.00, 0.00] | −0.695 [−0.75, −0.64] | <0.001 | 56.93 | 32.06 |
| Demo/256px-s1@530 | ODE 50 | 0.040 [0.00, 0.10] | −0.380 [−0.54, −0.22] | <0.001 | – | – | – | – | – |
| Demo/256px-s1 | ODE 200 | 0.670 [0.62, 0.71] | +0.275 [+0.14, +0.40] | <0.001 | 0.000 [0.00, 0.00] | −0.700 [−0.77, −0.63] | <0.001 | 58.83 | 34.60 |
| Demo/256px-s2 | ODE 200 | 0.755 [0.70, 0.81] | +0.360 [+0.22, +0.49] | <0.001 | 0.000 [0.00, 0.00] | −0.700 [−0.77, −0.63] | <0.001 | 59.01 | 31.58 |
| Demo/128px-fine@100 | ODE 200 | – | – | – | 0.190 [0.12, 0.27] | −0.510 [−0.60, −0.42] | <0.001 | – | 8.46 |
| Demo/128px-fine@200 | ODE 50 | – | – | – | 0.000 [0.00, 0.00] | −0.720 [−0.82, −0.62] | <0.001 | – | 47.87 |
| Demo/128px-fine@300 | – | – | – | – | – | – | – | – | 46.94 |
| Demo/128px-fine@400 | – | – | – | – | – | – | – | – | 41.97 |
| Demo/128px-fine@500 | – | – | – | – | – | – | – | – | 41.48 |
| Demo/128px-fine@530 | – | – | – | – | – | – | – | – | 41.52 |
| Demo/256px-fine@100 | ODE 50 | 0.080 [0.02, 0.16] | −0.340 [−0.50, −0.18] | <0.001 | 0.140 [0.04, 0.26] | −0.580 [−0.72, −0.42] | <0.001 | – | 6.96 |
| Demo/256px-fine@200 | ODE 50 | 0.000 [0.00, 0.00] | −0.420 [−0.58, −0.26] | <0.001 | 0.000 [0.00, 0.00] | −0.720 [−0.82, −0.62] | <0.001 | – | 35.03 |
| Demo/256px-fine@300 | – | – | – | – | – | – | – | – | 35.06 |
| Demo/256px-fine@400 | – | – | – | – | – | – | – | – | 34.73 |
| Demo/256px-fine@500 | – | – | – | – | – | – | – | – | 34.96 |
| Demo/256px-fine@530 | – | – | – | – | – | – | – | – | 36.23 |
| *Demo+Own* | | | | | | | | | |
| Demo+Own/own-replay@530 | ODE 200 | – | – | – | 0.115 [0.07, 0.15] | −0.585 [−0.67, −0.51] | <0.001 | – | – |
| Demo+Own/own-replay@1060 | ODE 200 | – | – | – | 0.610 [0.55, 0.68] | −0.090 [−0.17, −0.01] | 0.063 | – | – |
| Demo+Own/own-replay | ODE 400 | 0.688 [0.64, 0.73] | +0.333 [+0.24, +0.42] | <0.001 | 0.700 [0.64, 0.75] | +0.005 [−0.07, +0.07] | 0.937 | 46.85 | 14.58 |
| Demo+Own/demo-replay | ODE 400 | 0.623 [0.56, 0.68] | +0.268 [+0.16, +0.37] | <0.001 | 0.595 [0.54, 0.65] | −0.100 [−0.17, −0.03] | 0.003 | 45.24 | 15.84 |

## G Replication at full precision

All runs in the main text trained in bfloat16 without fp32 master weights (Appendix B). We repeated three conditions of the bowl-stacking pair with parameters and optimizer states in fp32 (TF32 matrix multiplies, micro-batch 8 with 8-step gradient accumulation, otherwise identical data, seed, learning rate, budget, and evaluation protocol; one seed per condition). After training, 100% of the expert's attention and MLP entries differ from Start, and 77.5–86.4% still differ after the bf16 cast applied at deployment. With trainable weights every condition learns its task more, and the contrast of Section 4.1 is unchanged: demonstrations erase the prior skill (every initial state worse at step 2118; 0.19 at step 530 against 0.00 in bf16), while own rollouts and on-policy RL keep it. RL now learns the new task ($+0.12$, $p = 0.018$) while retaining the prior one. Displacement keeps the paper's ordering, RL $<$ Own $\ll$ Demo, with a Demo/Own ratio of 3.7 against 4.1 in bf16; both SFT conditions displace less in action space than their bf16 counterparts despite fivefold larger weight updates.

Table 10: **fp32-master-weight replication.** One seed per condition; 200 episodes per cell (25 initial states $\times$ 8 noise seeds, deterministic sampler); $\Delta$ vs. Start paired on identical (state, seed) cells with a 95% CI and exact McNemar $p$; displacement is task-8 chunk MSE vs. Start $\times 10^{-3}$; "deployed changed" is the fraction of the expert's bf16 attention/MLP entries that differ from Start after the bf16 deployment cast. Start's reference values are its state-set-1 cells, the episodes these cells pair with; Table 1 reports Start over both state sets.

| arm (fp32 master) | task 64 (new): success, $\Delta$ vs. Start [95% CI] | task 8 (stage-1): success, $\Delta$ vs. Start [95% CI] | displ. | deployed changed | bf16 ref. (64 / 8) |
|---|---|---|---|---|---|
| RL/5it | 0.510, $+0.115$ [$+0.025$, $+0.205$], $p = 0.018$ | 0.690, $-0.010$ [$-0.096$, $+0.076$], $p = 0.91$ | 1.11 | 77.5% | 0.350 / 0.720 |
| Own/rl-rollouts @2118 | 0.545, $+0.150$ [$+0.068$, $+0.232$], $p = 0.0006$ | 0.785, $+0.085$ [$+0.002$, $+0.168$], $p = 0.060$ | 4.86 | 81.7% | 0.455 / 0.755 |
| Demo/256px @2118 | 0.895, $+0.500$ [$+0.425$, $+0.575$], $p < 10^{-4}$ | 0.045, $-0.655$ [$-0.722$, $-0.588$], $p < 10^{-4}$ | 18.2 | 86.4% | 0.720 / 0.000 |
| Demo/256px @530 | – | 0.190, $-0.510$ [$-0.587$, $-0.433$], $p < 10^{-4}$ | 14.9 | 80.0% | 0.015 / 0.000 |
| Start (reference) | 0.395 | 0.700 | 0 | 0% | – |

**Deviations from the main-text recipe.** The replication keeps the paper's data, seed, optimizer (AdamW, learning rate $5 \times 10^{-6}$, betas 0.9/0.95, weight decay 0.01, gradient clipping 1.0), global batch of 64, 2118 SFT steps with exports at steps 530 and 2118, and 5 GRPO iterations of 64 rollouts, and differs only in how the parameters are stored and updated. All parameters are kept in fp32 (the loader's cast to bf16 is undone after loading the bf16 Start checkpoint, which is lossless), so AdamW keeps fp32 master weights. Computation is full fp32 with TF32 matrix multiplies rather than bf16: the bf16 mixed-precision variant failed on dtype mismatches inside openpi's image augmentation and flow-matching code, so no bf16 compute was used. Because fp32 activations need more memory, the micro-batch is 8 (SFT) and 4 (GRPO) instead of 16, with gradient accumulation keeping the optimizer batch at 64; step time is 10.5 s against 6.0 s. The GRPO run uses RLinf's dense weight syncer instead of the sparse patch syncer, whose changed-entry patch exceeds GPU memory once nearly all entries change, and was resumed once from its iteration-1 checkpoint after that out-of-memory error. Exports are fp32 safetensors and are evaluated through the unchanged evaluation loader, which casts them to bf16 at load exactly as for the paper's checkpoints. The Demo step-530 new-task cell, the Own step-530 cells, and all state-set-2 cells were not run.

## H Off-policyness of the training targets

For every frame of the 256 px demonstration set (50 episodes, 11,734 frames) and of a 38-episode whole-episode subsample of Own/rl-rollouts' training set matched to it in frame count (11,786 frames), Start was given the frame's own observation (both camera images, state, and task prompt) and sampled an action chunk with the stochastic training sampler (Flow-SDE, $\sigma = 0.5$; two draws, averaged). The measure is the MSE between that chunk and the chunk stored in the dataset at the same frame, over the policy's 10-step action horizon and seven action dimensions, in the unnormalized policy-output units and chunk shape of the paper's displacement metric, $\times 10^{-3}$; 95% CIs are a bootstrap over episodes. The resampling floor is the MSE between Start's two draws at the same

frame, Start's own sampling noise. The deterministic sampler (ODE, 10 steps) gives the same means (Table 11).

Table 11: **Distance of the training targets from Start's own actions.** Top: mean chunk MSE ($\times 10^{-3}$) between the dataset's stored action chunk and Start's prediction on the dataset's own observation. Bottom: the same quantity split into the six arm channels pooled versus the gripper, and by episode third (reach, manipulate, place).

| training set | episodes / frames | mean [95% CI] | median | resampling floor | ODE sampler, mean |
|---|---|---|---|---|---|
| Human demos (Demo/256px) | 50 / 11,734 | 148.8 [143.4, 154.2] | 66.0 | 24.4 | 148.4 |
| Self-generated (Own/rl-rollouts) | 38 / 11,786 | 30.1 [27.1, 32.9] | 4.0 | 15.8 | 29.6 |
| ratio Demo / Own | | 5.0 | 16.4 | | 5.0 |

| training set | arm (6 dims) | gripper | first third | middle third | last third |
|---|---|---|---|---|---|
| Human demos | 67.1 [63.9, 70.4] | 638.8 [603.3, 674.7] | 85.9 | 265.5 | 94.9 |
| Self-generated | 8.1 [7.5, 8.7] | 162.0 [142.9, 180.3] | 24.1 | 42.8 | 23.3 |
| ratio Demo / Own | 8.3 | 3.9 | 3.6 | 6.2 | 4.1 |

# I Training sampler versus evaluation decoding

**Own-rollout SFT improves the prior skill.** Training on bowl-stacking successes, with no prior-task data, raised prior-task success in all six Own runs (pooled $\Delta$ =+0.07 [+0.02, +0.12]). The gain coincides with closing a gap between samplers: at Start, the prior task succeeds 0.88 of the time under the stochastic training sampler (Flow-SDE [35]; Section 3) but 0.69 under deterministic decoding, and after Own the two nearly agree (0.84 and 0.80; Table 12). Training on successes the stochastic sampler produced appears to carry some of its competence into the deterministic policy. We have not identified the full mechanism.

Table 12: **Evaluation decoding vs. the training sampler.** Success under evaluation decoding (ODE, 10 steps; 25 states $\times$ 8 draws per state set, 200 or 400 episodes) versus the training sampler (Flow-SDE $\sigma = 0.5$; 25 states $\times$ 2 draws, 50 episodes) for the main checkpoints. 95% bootstrap CI over initial states.

| | Task 64 (new) | | Task 8 (prior) | |
|---|---|---|---|---|
| Policy | ODE | SDE | ODE | SDE |
| Start | 0.355 [0.27, 0.44] | 0.360 [0.22, 0.50] | 0.695 [0.64, 0.75] | 0.880 [0.80, 0.96] |
| RL/5it | 0.355 [0.28, 0.44] | 0.360 [0.22, 0.50] | 0.733 [0.68, 0.78] | 0.720 [0.60, 0.84] |
| Own/start-rollouts | 0.345 [0.28, 0.42] | 0.480 [0.32, 0.64] | 0.802 [0.76, 0.84] | 0.840 [0.74, 0.92] |
| Demo/128px | 0.492 [0.44, 0.55] | 0.480 [0.34, 0.62] | 0.000 [0.00, 0.00] | 0.000 [0.00, 0.00] |

# J Task rollouts

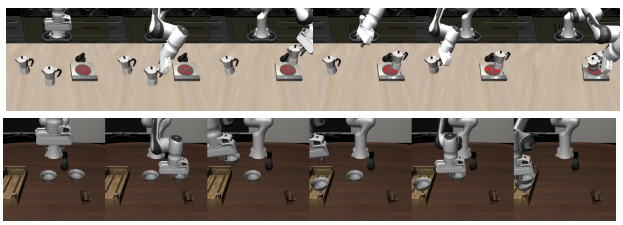


Figure 4: The prior task (top: put both moka pots on the stove, LIBERO-10) and the first new task (bottom: stack the right bowl on the left bowl and place them in the tray, LIBERO-90), one successful rollout of Start each under the training sampler, six evenly spaced frames from start to success.